2026-7-12 Linux进程--切换和调度

- 1. 从进程上下文理解切换
  - 1.1 寄存器与进程上下文
    - 1.1.1 寄存器硬件共享，寄存器内容属于进程私有
    - 1.1.2 上下文包含什么
    - 1.1.3 进程切换的本质
  - 1.2 什么情况下会发生切换
    - 1.2.1 时间片结束
    - 1.2.2 进程阻塞或主动让出
    - 1.2.3 更高优先级任务需要运行
  - 1.3 从并发与并行理解切换
    - 1.3.1 竞争性来自有限 CPU
    - 1.3.2 独立性保证进程互不干扰
    - 1.3.3 并发与并行不是同一个概念
- 2. 进程如何被组织成调度结构
  - 2.1 为什么调度需要数据结构
    - 2.1.1 选择前必须先找到可运行进程
    - 2.1.2 task_struct 需要进入多种队列
  - 2.2 Linux 的内嵌链表节点
    - 2.2.1 链表节点嵌入对象内部
    - 2.2.2 为什么从成员地址能找回整个对象
    - 2.2.3 一个进程可以进入多条链
    - 2.2.4 组织方式不只有链表
- 3. 调度与优先级
  - 3.1 调度是在可运行集合中选择
    - 3.1.1 调度的本质是选择
    - 3.1.2 每个 CPU 有自己的运行队列
  - 3.2 优先级不是权限
    - 3.2.1 权限决定能不能，优先级决定先后
    - 3.2.2 PRI 与 nice
    - 3.2.3 为什么 nice 只有 40 个级别
    - 3.2.4 两种数值范围不要混淆
    - 3.2.5 查看和修改 nice
- 4. Linux 2.6 O(1) 调度器的教学模型
  - 4.1 优先级数组
    - 4.1.1 每个优先级对应一个队列

        - 4.1.2 优先级数组的结构
    - 4.2 位图怎样加快选择
        - 4.2.1 不再逐个检查 140 个队列
        - 4.2.2 位运算适合快速判断
    - 4.3 active 与 expired
        - 4.3.1 活动队列
        - 4.3.2 过期队列
        - 4.3.3 为什么这样能缓解饥饿
    - 4.4 为什么称为 O(1)
        - 4.4.1 选择成本不随进程数量线性增长
        - 4.4.2 这是 Linux 2.6 的历史模型
- 5. 切换与调度的统一理解
    - 5.1 调度和切换的分工
    - 5.2 核心认识
    - 5.3 自检问题

本篇范围

前置阅读：2026-7-7 Linux进程--基础概念、2026-7-12 Linux进程--理解、2026-7-12 Linux进程--状态。本篇只整理进程上下文切换、进程组织、调度基础、优先级，以及 Linux 2.6 O(1) 调度器的教学模型。现代 Linux 的具体调度实现属于版本演进，不是本篇的主要讨论对象。

1. 从进程上下文理解切换

1.1 寄存器与进程上下文

1.1.1 寄存器硬件共享，寄存器内容属于进程私有

CPU 寄存器是硬件资源。某个 CPU 核心在执行代码时，使用的是同一套物理寄存器；但寄存器中保存的指令地址、临时结果和函数调用现场，是当前进程自己的运行数据。

因此要区分两件事：

```
寄存器硬件：CPU 所有执行流共享
寄存器中的数据：当前进程私有的执行现场
```

当进程离开 CPU 时，寄存器的值不能继续留在硬件中任其被下一个进程覆盖。操作系统需要把当前值保存起来，形成该进程的一份上下文；下一次恢复该进程时，再把这份上下文装回寄存器。

> 寄存器是共享的，但寄存器里保存的数据本质上属于进程私有，这份数据称为进程上下文。

1.1.2 上下文包含什么

一个进程的上下文至少要包括能够恢复执行现场的 CPU 状态：

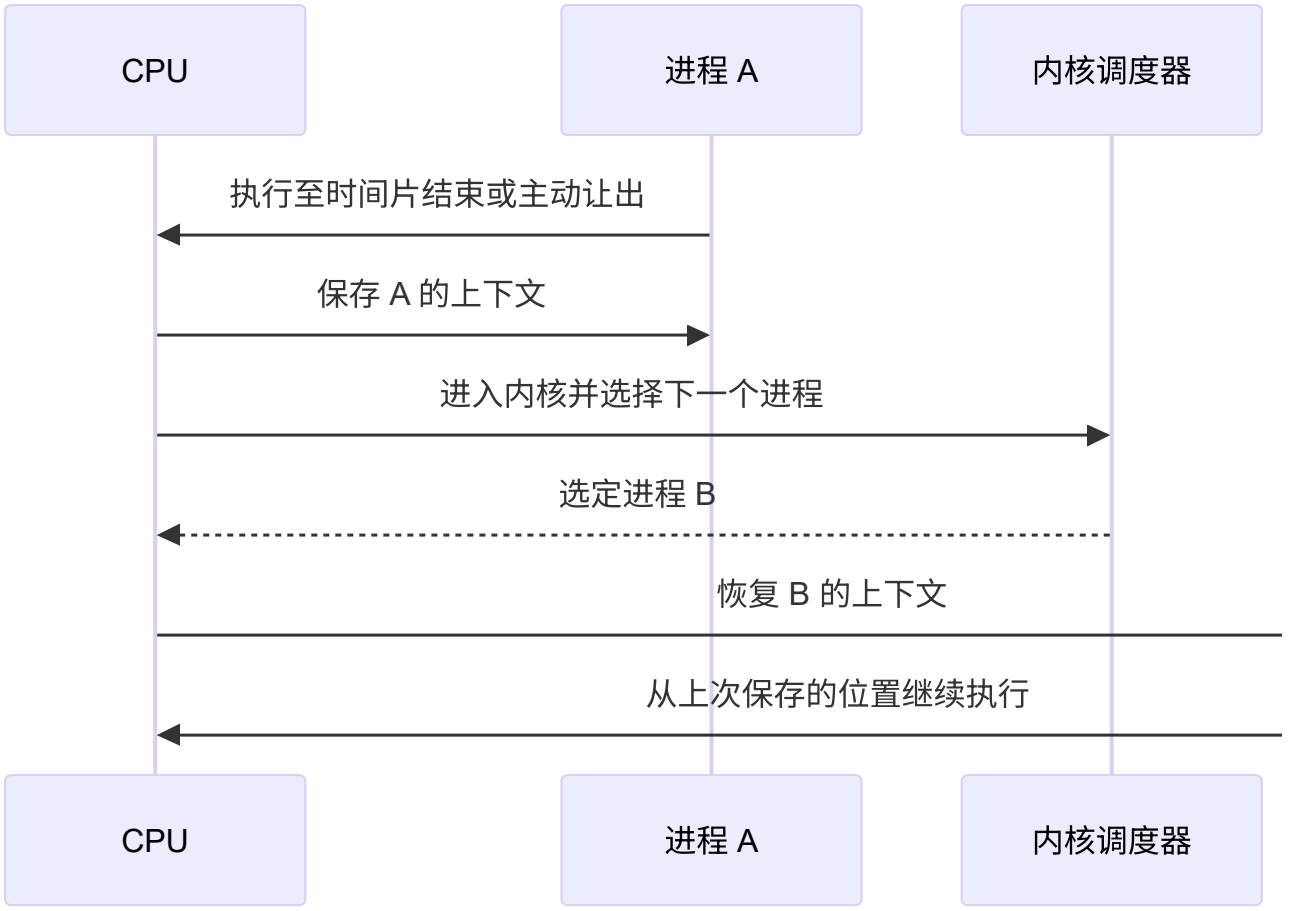
- 程序计数器：下一条要执行的指令地址；
- 通用寄存器：当前计算中的临时数据；
- 栈指针：当前函数和局部变量所在的位置；
- 状态寄存器：CPU 标志位等运行信息。

不同体系结构还会保存其他必要寄存器。上下文的目的不是保存进程在内存中的全部代码和数据，而是保存“重新进入 CPU 后怎样接着执行”的信息。

1.1.3 进程切换的本质

进程切换可以概括为：

```
保存旧进程现场 -> 选择新进程 -> 恢复新进程现场 -> 新进程继续执行
```



切换完成后，进程没有从头开始，而是从程序计数器和寄存器恢复出的位置继续执行。

1.2 什么情况下会发生切换

1.2.1 时间片结束

分时系统会让进程运行一个有限的时间片。时间片到期后，即使进程代码还没有执行完，操作系统也可以把它从 CPU 上取下，让其他进程运行。

这是**抢占式调度**的重要体现：进程不需要主动调用某个“让出 CPU”的函数，操作系统也能在合适的时机制止它继续运行。

1.2.2 进程阻塞或主动让出

进程等待 I/O、等待锁或等待其他事件时，无法继续推进，继续占着 CPU 没有意义。操作系统会把它移出运行队列，并选择其他可运行进程。

进程也可以主动让出 CPU，让同优先级或更适合运行的进程先执行。

1.2.3 更高优先级任务需要运行

当有更重要的任务变为可运行状态时，操作系统可能暂停当前任务，转向更高优先级任务。具体是否会立即抢占，取决于内核版本、调度类和当前配置。

此外，当前进程结束或发生异常时，也需要重新选择下一个进程。

1.3 从并发与并行理解切换

1.3.1 竞争性来自有限 CPU

系统中可运行的进程数量通常多于 CPU 核心数量，因此进程会竞争 CPU 时间。优先级和调度策略就是为了决定竞争顺序。

1.3.2 独立性保证进程互不干扰

多个进程运行时应拥有独立的资源视图。进程切换不能导致一个进程直接修改另一个进程的私有上下文和数据。

1.3.3 并发与并行不是同一个概念

概念	含义
并发	多个任务在同一时间段内交替推进，可能在同一个 CPU 上切换
并行	多个任务在同一时刻由多个 CPU 核心同时执行

单核 CPU 上的并发主要依靠快速切换；多核系统可以同时存在并发和并行。

2. 进程如何被组织成调度结构

2.1 为什么调度需要数据结构

2.1.1 选择前必须先找到可运行进程

调度器最重要的动作是“选择下一个进程”：

> 从所有可运行进程中选出一个最适合当前 CPU 的进程

内核不可能在每次选择时扫描并重新构造所有进程对象，因此需要在 `task_struct` 之间维护高效的组织结构。

2.1.2 task_struct 需要进入多种队列

一个进程可能同时在多种管理中：

- 在全局任务链表中，便于遍历所有进程；
- 在 CPU 运行队列中，等待被执行；
- 在设备等待队列中，等待某个事件；
- 在其他定时器或资源管理结构中。

如果每个队列都复制一份进程对象，会浪费内存，也会让多个副本难以保持一致。Linux 因此采用内嵌链表节点的方式组织同一个 `task_struct`。

2.2 Linux 的内嵌链表节点

2.2.1 链表节点嵌入对象内部

最基础的链表节点可以简化为：

```
struct list_head {
    struct list_head *next;
    struct list_head *prev;
};
```

Linux 内核会把所有 `task_struct` 组织到全局任务链表中；运行队列和等待队列又可以通过额外的内嵌节点组织同一批对象。

关键点不是把整个 `task_struct` 放进链表节点，而是把一个 `list_head` 成员内嵌到 `task_struct` 中。下面的字段名只是教学示意，具体结构会随内核版本变化：

```
struct task_struct {
    /* 其他进程属性 */
    struct list_head tasks;
```

```
    struct list_head run_list;
    struct list_head wait_list;
};
```

链表实际连接的是 `tasks`、`run_list` 等成员，而不是 `task_struct` 对象本身的首地址。

2.2.2 为什么从成员地址能找回整个对象

C 语言中，一个变量的地址是它占用多个字节中的最小地址。结构体成员的地址与结构体起始地址之间存在固定偏移。

例如：

```
struct Example {
    int a;
    int b;
    int c;
};
```

若知道 `c` 的地址，就可以减去 `c` 在结构体中的偏移，得到整个结构体的起始地址：

```
结构体地址 = 成员地址 - 成员偏移量
```

Linux 用 `offsetof` 求偏移，用 `container_of` 从成员找回所属对象。可以简化理解为：

```
#define container_of(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))
```

所以调度器看到一个链表节点时，可以通过它找到完整的 `task_struct`，进而访问 PID、状态、优先级等其他属性。

2.2.3 一个进程可以进入多条链

因为不同链表使用 `task_struct` 中不同的内嵌节点，同一个进程对象可以：

```
挂在全局 task 链表上
同时挂在某个 CPU 的运行队列上
```

同时挂在某个设备的等待队列上

这正是“内嵌节点”的扩展性：链表代码只需要维护一份，任何包含 `list_head` 的结构都可以复用同一套链表操作。

2.2.4 组织方式不只有链表

链表适合维护顺序和遍历关系，但不是所有场景都适合线性查找。内核也会使用数组、队列、哈希表、红黑树等结构。

选择数据结构的原则仍然是：

> 先根据要解决的问题描述对象，再选择能让查找、插入、删除和调度代价合理的数据结构。

3. 调度与优先级

3.1 调度是在可运行集合中选择

3.1.1 调度的本质是选择

当 CPU 空闲或当前进程需要让出 CPU 时，内核调度器要从可运行进程中选出下一个执行者。

调度的目标是兼顾：

- 正确性：只能选择确实具备运行条件的进程；
- 效率：选择过程不能引入过大的系统开销；
- 公平性：普通进程不能长期得不到 CPU；
- 响应性：重要任务需要及时得到处理；
- 吞吐量：减少无效切换和缓存抖动。

这些目标可能互相冲突，不同调度策略是权衡的结果。

3.1.2 每个 CPU 有自己的运行队列

在经典模型中，每个 CPU 都有一个运行队列，用 `struct rq` 管理本 CPU 的可运行任务。

多个 CPU 之间还要考虑负载均衡：如果某个 CPU 的运行队列特别拥挤，而另一个 CPU 空闲，就需要迁移任务。

3.2 优先级不是权限

3.2.1 权限决定能不能，优先级决定先后

概念	解决的问题
权限	进程能否访问某个资源
优先级	具备访问资格后，谁先获得有限资源

一个进程没有访问资源的权限，优先级再高也不能越权；对于可以竞争的 CPU 时间，优先级决定执行的先后和份额。

3.2.2 PRI 与 nice

通过：

```
ps -l
```

可以观察进程的 `PRI` 和 `NI` 等字段。

- `PRI`：显示优先级，数值通常越小，越早获得执行机会；
- `NI`：nice 值，是对优先级的修正量。

可以近似写成：

```
显示优先级 PRI ≈ 基准值 + nice
```

`nice` 的范围是 `-20 ~ 19`，数值越小，表示进程越“不客气”，优先级越高；数值越大，表示进程越“谦让”，优先级越低。

3.2.3 为什么 nice 只有 40 个级别

nice 的范围限制来自内核调度设计：

- 需要提供足够的优先级粒度；
- 不能让普通用户随意给进程极高优先级；
- 调整优先级会影响排队和调度，因此不应成为高频操作。

普通用户通常只能增大 nice 值，也就是降低自己进程的优先级；提高优先级、使用更小的 nice 值通常需要管理员权限。

3.2.4 两种数值范围不要混淆

课堂中会出现两套相关但不同的数字：

场景	范围	说明
`ps` 中显示	常见为 `60 ~ 99`	基准约 80，叠加 `-20 ~ 19` 的 nice
Linux 2.6 内核普通进程优先级	`100 ~ 139`	内核内部优先级，数字越小优先级越高
Linux 2.6 内核实时优先级	`0 ~ 99`	实时进程范围

因此不能简单把 `ps` 显示的 `PRI` 与内核数组下标当成同一个数值体系。

3.2.5 查看和修改 nice

查看进程优先级：

```
ps -l
```

以指定 nice 值启动程序：

```
nice -n 10 ./myprocess
```

调整已运行的进程：

```
renice 10 -p <PID>
```

还可以在 `top` 中按 `r`，输入 PID 和新的 nice 值进行调整。

调整界面中出现的 `old priority` 或 `old PRI`，指的是修改前当前显示的优先级值，不是某个固定的内核基准值。修改后会让进程按新的 nice 修正重新参与调度排队。

频繁大幅修改优先级不仅会造成调度不连续，也会增加调度器重新排队的成本，因此这不是正常业务逻辑应高频执行的操作。

4. Linux 2.6 O(1) 调度器的教学模型

4.1 优先级数组

4.1.1 每个优先级对应一个队列

Linux 2.6 的 O(1) 调度器用优先级作为数组下标：

```
queue[0]   -> 优先级 0 的进程队列
queue[1]   -> 优先级 1 的进程队列
...
queue[139] -> 优先级 139 的进程队列
```

普通进程使用 `100 ~ 139`，实时进程使用 `0 ~ 99`。下标越小，优先级越高。

同一优先级的多个进程组成 FIFO 队列：

```
先进入该优先级队列的进程先被选择
```

所以一次调度可以先通过优先级找到队列，再从该队列取第一个进程；优先级选择类似按编号索引，而不是逐个比较所有进程。

4.1.2 优先级数组的结构

教学模型中的优先级数组可简化为：

```
struct prio_array {
    unsigned int nr_active;     /* 当前队列中可运行进程数 */
    unsigned long bitmap[5];    /* 140 个优先级是否存在进程 */
```

```
    struct list_head queue[140];/* 每个优先级一条 FIFO 链表 */
};
```

`bitmap` 用每个二进制位表示对应优先级队列是否为空，5 个 32 位整数可以提供 160 位，足够覆盖 140 个优先级。

4.2 位图怎样加快选择

4.2.1 不再逐个检查 140 个队列

如果每次从 `queue[0]` 一路遍历到 `queue[139]`，虽然上限固定，但选择成本仍然较高。位图把所有队列的空闲情况压缩成若干整数：

```
某位为 0 -> 对应优先级队列为空
某位为 1 -> 对应优先级队列非空
```

调度器只需要找到最低的置位比特，就能确定当前最高优先级的非空队列。

4.2.2 位运算适合快速判断

位图可以用较少的内存和固定的位运算完成：

- 判断是否有可运行进程；
- 快速定位最高优先级非空队列；
- 避免对 140 条链表逐个线性扫描。

找到队列后，再取该 FIFO 链表的第一个节点，并通过内嵌链表节点恢复对应的 `task_struct`。

4.3 active 与 expired

4.3.1 活动队列

每个 CPU 的 O(1) 运行队列维护两套优先级数组：

```
active  -> 当前活动队列
expired -> 过期队列
```

活动队列中的进程仍拥有时间片，调度器会按照优先级从 active 中选择进程。

4.3.2 过期队列

当进程时间片耗尽时，它不是简单回到原队列重新排队，而是被放入 expired 队列。等 active 中的任务处理到一定程度后，操作系统交换两个队列的角色：

```
active  <-> expired
```

原来的 expired 成为新的 active，进程时间片会被重新计算。

4.3.3 为什么这样能缓解饥饿

假设一批低优先级进程正在 active 中运行，此时不断创建更高优先级的新进程。如果新进程总插到 active 最前面，旧进程可能一直得不到 CPU。

O(1) 调度器的处理方式是：新进入或时间片耗尽的进程进入 expired，等当前 active 处理完后再统一交换。这样，旧任务不会因为源源不断的新任务而永远排在后面。

这并不等于任何平台、任何配置都绝对不存在延迟或饥饿，但该双队列结构确实为普通任务提供了批次轮转的机会。

4.4 为什么称为 O(1)

4.4.1 选择成本不随进程数量线性增长

在这个模型中：

- 优先级总数固定为 140；
- 位图长度固定；
- 找到最高优先级非空队列是固定规模的位操作；
- 取 FIFO 队首是固定规模操作。

因此，单纯“从运行队列中选出下一个进程”的算法代价可以视为常数，与系统中进程总数没有线性关系，所以称为 O(1) 调度算法。

这里的 O(1) 只描述调度选择的结构性代价，不表示一次真实的上下文切换没有成本。寄存器保存、页表变化、缓存和 TLB 影响等仍然会带来实际开销。

4.4.2 这是 Linux 2.6 的历史模型

O(1) 调度器是理解优先级数组、位图和 active/expired 双队列的优秀教学模型，但现代 Linux 已经采用后续的 CFS、EEVDF 等调度机制。

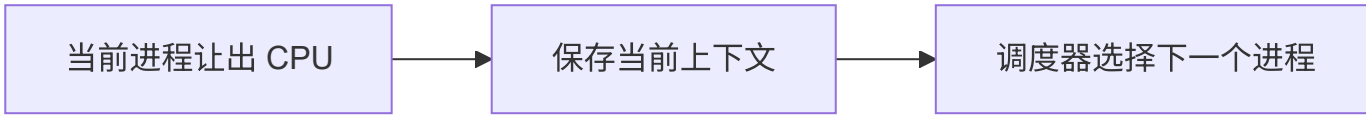
因此本篇的安排是：

```
用 Linux 2.6 O(1) 模型学习调度结构
        ↓
理解优先级、队列、位图、公平轮转
        ↓
后续再区分现代 Linux 的实际调度类与算法
```

不要把“Linux 当前等于 O(1) 调度器”作为结论。Linux 同时支持实时类和普通类任务，普通任务的调度目标偏向公平与响应性，具体机制随内核版本演进。

5. 切换与调度的统一理解

5.1 调度和切换的分工



- **调度**解决“下一个选谁”；
- **切换**解决“怎样把旧现场换成新现场”。

调度策略决定选择结果，上下文切换完成执行流的交接。

5.2 核心认识

1. 寄存器硬件共享，但寄存器内容作为进程上下文私有保存。

2. 进程切换的核心是保存旧上下文、恢复新上下文。
3. 时间片、阻塞和抢占都可能触发切换。
4. `task_struct` 通过内嵌链表节点进入多种管理结构。
5. 调度器从可运行队列中选择下一个进程，优先级影响选择顺序。
6. 链表操作只需要维护嵌入节点，再通过偏移就能找回完整的 `task_struct`。
7. Linux 2.6 用优先级数组、位图和 active/expired 双队列实现 O(1) 调度选择。
8. O(1) 选择不等于零成本切换，也不等于现代 Linux 仍使用同一调度器。

5.3 自检问题

应该能够区分这些概念

1. 寄存器硬件和寄存器中的进程上下文为什么不是一回事?
2. 上下文切换需要保存哪些信息，为什么不必保存整个进程地址空间?
3. 为什么同一个 `task_struct` 能同时出现在全局链表和运行队列中?
4. `container_of` 为什么能从内嵌节点找回完整结构体?
5. 权限、PRI 和 nice 分别解决什么问题?
6. `ps` 中的 PRI 范围为什么不能直接当作 Linux 2.6 内核优先级数组下标?
7. active/expired 双队列怎样缓解旧进程长期得不到 CPU?
8. “调度选择是 O(1)”为什么不表示真实进程切换没有成本?